

# Web Technology

*# This is an example of a simple Python program that uses web technology to make a GET request*

```
import requests

url = 'https://api.example.com/data'
response = requests.get(url)

if response.status_code == 200:
    data = response.json()
    print(data)
else:
    print('An error occurred:', response.status_code)
```

## Unit 1 - Introduction to Web Technology

Web technology refers to the means by which computers communicate with each other using markup languages and multimedia packages. It provides a way to interact with hosted information, like websites. Web technology involves the use of hypertext markup language (HTML) and cascading style sheets (CSS).

### Introduction to Web Technology

Web technology refers to the means by which computers communicate with each other using markup languages and multimedia packages. It gives us a way to interact with hosted information, like websites. Web technology involves the use of hypertext markup language (HTML) and cascading style sheets (CSS).

### Web Development Strategies

Web development strategies refer to the methods and techniques used to create, design, and maintain a website. Here are some key strategies to consider when developing a website:

1. **Define the purpose and goals of the website.** Before starting the development process, it is important to clearly define the purpose and goals of the website. This will help guide the design and development process and ensure that the website meets the needs of its intended audience.
2. **Choose the right technology stack.** The technology stack refers to the combination of programming languages, frameworks, and tools used to build a website. It is important to choose the right technology stack that is suitable for the website's requirements and goals.
3. **Design for the user experience.** The user experience (UX) refers to the overall experience a user has when interacting with a website. It is

important to design the website with the user experience in mind, ensuring that it is easy to use, navigate, and understand.

4. **Ensure the website is responsive and mobile-friendly.** With the increasing use of mobile devices to access the internet, it is important to ensure that the website is responsive and mobile-friendly. This means that the website should be designed to adapt to different screen sizes and devices, providing a seamless user experience across all platforms.
5. **Implement search engine optimization (SEO) techniques.** Search engine optimization (SEO) refers to the process of improving the visibility of a website in search engine results pages. By implementing SEO techniques, the website can rank higher in search results, increasing its visibility and attracting more traffic.
6. **Regularly update and maintain the website.** A website is not a one-time project, but rather an ongoing process. It is important to regularly update and maintain the website, ensuring that it remains relevant, up-to-date, and functional.

These are just a few of the many strategies that can be used when developing a website. By considering these strategies and others, a website can be designed and developed to meet the needs of its intended audience and achieve its goals.

## History of Web

- The World Wide Web (WWW) was invented in 1989 by a British scientist named Tim Berners-Lee while working at CERN .
- The Web was originally conceived and developed to meet the demand for automated information-sharing between scientists in universities and institutes around the world .
- Tim Berners-Lee and his colleagues at CERN created a protocol, Hyper-Text Transfer Protocol (HTTP), which standardized communication between servers and clients .
- By the end of 1990, the first web page was served on the open internet, and in 1991, people outside of CERN were invited to join this new web community .
- As the web began to grow, Tim realized that its true potential would only be unleashed if anyone, anywhere could use it without paying a fee or having to ask for permission .
- Yahoo! Search, launched the same year, was the first popular search engine on the World Wide Web .
- Online shopping began to emerge with the launch of Amazon's shopping site by Jeff Bezos in 1995 and eBay by Pierre Omidyar the same year .

## History of Internet

- The internet got its start in the United States more than 50 years ago as a government weapon in the Cold War .
- For years, scientists and researchers used it to communicate and share data with one another .
- The origins of the internet are rooted in the USA of the 1950s .
- The Cold War was at its height and huge tensions existed between North America and the Soviet Union .
- Both superpowers were in possession of deadly nuclear weapons, and people lived in fear of long-range surprise attacks .
- Computer scientists Vinton Cerf and Bob Kahn are credited with inventing the Internet communication protocols we use today and the system referred to as the Internet .
- Before the current iteration of the Internet, long-distance networking between computers was first accomplished in a 1969 experiment by two research teams at UCLA and Stanford .
- The term “internet” was reflected in the first RFC published on the TCP protocol (RFC) .
- Internet, a system architecture that has revolutionized communications and methods of commerce by allowing various computer networks around the world to interconnect .
- Sometimes referred to as a “network of networks,” the Internet emerged in the United States in the 1970s but did not become visible to the general public until the early 1990s .

## Protocols Governing Web

1. **HTTP (Hypertext Transfer Protocol):** This is the protocol used for transmitting web pages over the Internet. It defines how messages are formatted and transmitted, and what actions Web servers and clients should take in response to various commands.
2. **HTTPS (Hypertext Transfer Protocol Secure):** This is a secure version of HTTP that uses SSL/TLS to encrypt data transmitted between the web server and the client.
3. **FTP (File Transfer Protocol):** This protocol is used for transferring files between computers on a network. It is commonly used for uploading files to a web server or downloading files from a server.
4. **TCP/IP (Transmission Control Protocol/Internet Protocol):** This is the basic communication protocol of the Internet. It is a set of rules that define how data is transmitted over a network.
5. **DNS (Domain Name System):** This is a protocol used for translating human-readable domain names (such as `www.example.com`) into IP addresses that computers can understand.

6. **SMTP (Simple Mail Transfer Protocol):** This protocol is used for sending email messages between servers.
7. **IMAP (Internet Message Access Protocol):** This protocol is used for retrieving email messages from a server.
8. **POP3 (Post Office Protocol version 3):** This protocol is used for retrieving email messages from a server and downloading them to a local client.
9. **SSL/TLS (Secure Sockets Layer/Transport Layer Security):** These protocols are used for encrypting data transmitted over a network. They are commonly used in combination with other protocols (such as HTTPS) to provide secure communication.

## Writing Web Projects

When writing web projects, there are several key points to keep in mind:

1. **Plan and organize your content:** Before you start writing, it's important to plan and organize your content. This includes defining the purpose of your website, identifying your target audience, and determining the structure and layout of your site.
2. **Write clear and concise content:** When writing for the web, it's important to keep your content clear and concise. This means using short sentences and paragraphs, avoiding jargon, and breaking up your content with headings and subheadings.
3. **Use proper formatting:** Proper formatting can help make your content more readable and engaging. This includes using headings, lists, and bold or italic text to highlight important information.
4. **Incorporate multimedia:** Including multimedia such as images, videos, and audio can help make your content more engaging and visually appealing.
5. **Optimize for search engines:** Search engine optimization (SEO) is the process of improving the visibility of your website in search engine results pages. This includes using keywords, meta tags, and other techniques to help search engines understand the content of your site.
6. **Proofread and edit:** Before publishing your content, it's important to proofread and edit it to ensure that it is free of errors and reads well.

By following these tips, you can create effective and engaging web content that will help you achieve your goals.

## Connecting to Internet

Here is an example of how to connect to the internet using Python:

```

import socket

def check_internet():
    try:
        socket.create_connection(("www.google.com", 80))
        return True
    except OSError:
        pass
    return False

if check_internet():
    print("Connected to the internet")
else:
    print("Not connected to the internet")

```

## Introduction to Internet services

Internet services refer to the services provided by the Internet, including the World Wide Web, email, file transfer protocol (FTP), and others. These services allow users to access and share information, communicate with others, and perform various tasks online.

One of the most widely used Internet services is the World Wide Web, which consists of a vast network of interconnected documents and other resources, linked by hyperlinks and URLs. Users can access the Web using a web browser, which retrieves and displays web pages and other content.

Email is another popular Internet service, allowing users to send and receive electronic messages. Email can be used for personal or business communication and can include text, images, and other attachments.

FTP is a standard network protocol used to transfer files from one host to another over the Internet. It is commonly used to upload files to a web server or to download files from a server to a local computer.

These are just a few examples of the many Internet services available. As technology continues to evolve, new services are being developed to meet the changing needs of users.

## Introduction to Internet tools

The internet is a vast network of interconnected computers and servers that allows for the exchange of information and communication. There are many tools available that make use of the internet to provide various services and functionalities. Some of the most commonly used internet tools include:

1. **Web browsers:** A web browser is a software application that allows users to access, view, and interact with websites and web pages. Some popu-

lar web browsers include Google Chrome, Mozilla Firefox, and Microsoft Edge.

2. **Search engines:** A search engine is a tool that helps users find information on the internet by searching for keywords and returning relevant results. Some popular search engines include Google, Bing, and Yahoo.
3. **Email:** Email is a tool that allows users to send and receive electronic messages over the internet. Some popular email providers include Gmail, Yahoo Mail, and Microsoft Outlook.
4. **Social media:** Social media refers to websites and applications that allow users to create and share content and interact with others. Some popular social media platforms include Facebook, Twitter, and Instagram.
5. **Cloud storage:** Cloud storage is a tool that allows users to store and access data over the internet. Some popular cloud storage providers include Google Drive, Dropbox, and Microsoft OneDrive.

These are just a few examples of the many internet tools available. These tools have revolutionized the way we communicate, access information, and share content, and continue to evolve and improve over time.

### Introduction to client-server computing

Client-server computing is a distributed computing model in which client applications request services from server processes. The server processes fulfill the requests and return the results to the client applications.

1. In this model, the client and server are separate entities, often running on different machines and communicating over a network.
2. The client is responsible for the user interface and user interaction, while the server is responsible for managing shared resources and providing services to the clients.
3. This separation of concerns allows for greater scalability, as multiple clients can access the same server resources concurrently.
4. It also allows for greater flexibility, as the client and server can be developed and maintained independently of each other.
5. Common examples of client-server systems include web applications, where the client is a web browser and the server is a web server, and database systems, where the client is a database application and the server is a database server.

### Core Java

Here is an example of a simple “Hello, World” program written in Core Java:

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

```
}  
}
```

## Introduction to Java

Java is a high-level, class-based, object-oriented programming language that is designed to have as few implementation dependencies as possible. It is a general-purpose programming language intended to let application developers write once, run anywhere (WORA), meaning that compiled Java code can run on all platforms that support Java without the need for recompilation.

Some key features of Java include:

1. **Platform independence:** Java programs can be easily moved from one computer system to another, anywhere and anytime. This is possible due to the platform-independent bytecode language produced by the Java compiler.
2. **Object-oriented:** Java is an object-oriented language, which means that it represents concepts as “objects” with “fields” (which are attributes that describe the object) and “methods” (actions that the object can perform).
3. **Robust and secure:** Java is designed to be a robust and secure language. It has strong memory management and automatic garbage collection, which helps prevent common programming errors and security vulnerabilities.
4. **Rich API:** Java has a rich set of APIs (Application Programming Interfaces) that provide a wide range of functionality, from basic data structures to advanced networking and security features.
5. **Large community:** Java has a large and active community of developers, which provides support, resources, and libraries for building applications in Java.

Java is widely used in various domains, including web development, mobile development, and enterprise applications. It is also a popular language for teaching computer science and programming concepts.

## Operator in Core Java

- Operators are special symbols that perform specific operations on one, two, or three operands, and then return a result.
- In Java, operators are categorized based on the number and type of operands they work on, as well as the type of operation they perform.
- The main categories of operators in Java are:
  - Arithmetic Operators: used to perform basic mathematical operations such as addition, subtraction, multiplication, and division.

- Relational Operators: used to compare two values and return a boolean result (true or false) based on whether the comparison is true.
- Bitwise Operators: used to perform operations on individual bits of integer values.
- Logical Operators: used to combine two or more boolean expressions and return a boolean result.
- Assignment Operators: used to assign a value to a variable.
- Conditional Operators: used to evaluate a boolean expression and return one of two values based on whether the expression is true or false.
- Unary Operators: used to perform an operation on a single operand.
- Each operator has a specific syntax and precedence, which determines the order in which operations are performed in an expression.
- It is important to understand the behavior and precedence of operators in order to write correct and efficient code in Java.

## Data type in Core Java

In Core Java, data types are used to define the type of data that a variable can hold. There are two types of data types in Java: primitive and non-primitive (also known as reference types).

1. **Primitive data types:** These are the most basic data types in Java and are used to hold simple values. There are eight primitive data types in Java: `byte`, `short`, `int`, `long`, `float`, `double`, `char`, and `boolean`. Each of these data types has a fixed size and range.
2. **Non-primitive data types:** These are also known as reference types and are used to hold objects. Non-primitive data types include classes, interfaces, and arrays. Unlike primitive data types, the size of non-primitive data types is not fixed and can vary depending on the data they hold.

It is important to choose the appropriate data type for a variable based on the data it will hold, as this can affect the performance and memory usage of the program. For example, using an `int` data type to hold a large number would be more efficient than using a `byte` data type, as the `int` data type has a larger range and can hold larger values.

## Variable in Core Java

A variable is a container that holds values that are used in a Java program. In order to use a variable, you must first declare it by specifying its data type and name. Here is an example of declaring a variable in Java:

```
int myVariable;
```

In this example, we have declared a variable named `myVariable` of type `int`. Once a variable is declared, you can assign a value to it using the assignment



operator `=`. Here is an example of assigning a value to a variable:

```
myVariable = 10;
```

In this example, we have assigned the value `10` to the variable `myVariable`. You can also declare and initialize a variable in a single statement, like this:

```
int myVariable = 10;
```

In this example, we have declared a variable named `myVariable` of type `int` and assigned it the value `10` in a single statement. Variables can be of different data types, such as `int`, `double`, `char`, `boolean`, and `String`. The data type of a variable determines the type of values it can hold. For example, an `int` variable can hold integer values, while a `String` variable can hold text values.

## Arrays in Core Java

An array is a collection of elements of the same type, stored in contiguous memory locations. Here is an example of how to declare, initialize, and access an array in Java:

```
int[] myArray = new int[5]; // declaration and initialization of an array of size 5
myArray[0] = 1; // assigning value to the first element
myArray[1] = 2; // assigning value to the second element
myArray[2] = 3; // assigning value to the third element
myArray[3] = 4; // assigning value to the fourth element
myArray[4] = 5; // assigning value to the fifth element

for (int i = 0; i < myArray.length; i++) { // accessing elements of the array
    System.out.println(myArray[i]);
}
```

## Methods & Classes in Core Java

```
public class MyClass {
    int x = 5;

    public static void main(String[] args) {
        MyClass myObj = new MyClass();
        System.out.println(myObj.x);
    }
}
```

In the above code, `MyClass` is a class in Core Java. A class is a blueprint for creating objects. It contains fields (variables) and methods (functions) that define the behavior and state of the objects created from it.

The `main` method is the entry point of the program. It is a static method, which means it can be called without creating an instance of the class.

The `myObj` variable is an instance of the `MyClass` class, created using the `new` keyword. The `x` field of the `myObj` object is accessed using the dot notation (`myObj.x`).

## Inheritance in Core Java

Inheritance is a mechanism in Java that allows one class to inherit the properties and behaviors of another class. This is achieved by using the `extends` keyword. Here is an example:

```
class Animal {
    public void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    public void bark() {
        System.out.println("Dog is barking");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat();
        dog.bark();
    }
}
```

In this example, the `Dog` class extends the `Animal` class, which means that the `Dog` class inherits the `eat` method from the `Animal` class. The `Dog` class also has its own method, `bark`. In the `main` method, we create an instance of the `Dog` class and call both the `eat` and `bark` methods on it. The output of this code will be:

```
Animal is eating
Dog is barking
```

## Package and Interface in Core Java

A package in Java is a way to group related classes and interfaces together. Packages provide a way to organize code and control access to classes and interfaces.

Here is an example of how to create a package and define an interface within it:

```
// Define a package named "myPackage"
package myPackage;
```

```
// Define an interface named "MyInterface" within the package
public interface MyInterface {
    // Define a method signature within the interface
    public void myMethod();
}
```

To use the interface defined in the package, it must be imported into the class that will implement it. Here is an example of how to do this:

```
// Import the interface from the package
import myPackage.MyInterface;

// Define a class that implements the interface
public class MyClass implements MyInterface {
    // Implement the method defined in the interface
    public void myMethod() {
        // Method implementation
    }
}
```

## Exception Handling in Core Java

Exception handling is a mechanism in Java to handle runtime errors and exceptional conditions. It is achieved using the `try`, `catch`, and `finally` blocks.

Here is an example of how to use exception handling in Core Java:

```
public class ExceptionExample {
    public static void main(String[] args) {
        try {
            int a = 5;
            int b = 0;
            int c = a / b;
            System.out.println("Result: " + c);
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero");
        } finally {
            System.out.println("This code will always be executed");
        }
    }
}
```

In the above example, we are trying to divide a number by zero, which will throw an `ArithmeticException`. The `catch` block catches this exception and handles it by printing an error message. The `finally` block is executed regardless of whether an exception is thrown or not.

## Multithread programming in Core Java

Multithreading in Java is a process of executing multiple threads simultaneously. A thread is a lightweight sub-process, the smallest unit of processing. Here is an example of how to create a thread in Java:

```
class MultithreadingDemo extends Thread {
    public void run() {
        try {
            System.out.println("Thread " + Thread.currentThread().getId() + " is running");
        } catch (Exception e) {
            System.out.println("Exception is caught");
        }
    }
}

public class Multithread {
    public static void main(String[] args) {
        int n = 8;
        for (int i = 0; i < n; i++) {
            MultithreadingDemo object = new MultithreadingDemo();
            object.start();
        }
    }
}
```

This code creates a class `MultithreadingDemo` that extends the `Thread` class and overrides its `run` method. The `run` method is where the code for the new thread is defined. In the `main` method, we create 8 instances of the `MultithreadingDemo` class and call their `start` method to start the new threads. When the `start` method is called, the `run` method of the corresponding `MultithreadingDemo` object is executed in a new thread.

## I/O in Core Java

```
import java.io.*;

public class IOExample {
    public static void main(String[] args) {
        try {
            // Create a new file
            File file = new File("example.txt");
            file.createNewFile();

            // Write to the file
            FileWriter writer = new FileWriter(file);
            writer.write("Hello, World!");
        }
    }
}
```

```

        writer.close();

        // Read from the file
        FileReader reader = new FileReader(file);
        BufferedReader bufferedReader = new BufferedReader(reader);
        String line = bufferedReader.readLine();
        while (line != null) {
            System.out.println(line);
            line = bufferedReader.readLine();
        }
        bufferedReader.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}

```

## Java Applet in Core Java

Here is an example of a simple Java Applet that displays “Hello, World!” on the screen:

```

import java.applet.Applet;
import java.awt.Graphics;

public class HelloWorld extends Applet {
    public void paint(Graphics g) {
        g.drawString("Hello, World!", 20, 20);
    }
}

```

To run this applet, you need to embed it in an HTML file and open it in a web browser that supports Java. Here is an example of an HTML file that embeds the above applet:

```

<html>
    <body>
        <applet code="HelloWorld.class" width="200" height="200">
        </applet>
    </body>
</html>

```

Save the Java code in a file named `HelloWorld.java` and compile it using the `javac` command. Then, save the HTML code in a file named `hello.html` and open it in a web browser. You should see the message “Hello, World!” displayed on the screen.

## String handling in Core Java

```

public class StringHandling {
    public static void main(String[] args) {
        String str = "Hello, World!";
        System.out.println("Original String: " + str);

        // Get the length of the string
        int length = str.length();
        System.out.println("Length of the string: " + length);

        // Concatenate two strings
        String str2 = " Have a nice day!";
        String concatenatedString = str.concat(str2);
        System.out.println("Concatenated String: " + concatenatedString);

        // Get a character at a specific index
        char ch = str.charAt(7);
        System.out.println("Character at index 7: " + ch);

        // Get the index of a character
        int index = str.indexOf('W');
        System.out.println("Index of character 'W': " + index);

        // Convert string to uppercase
        String upperCaseString = str.toUpperCase();
        System.out.println("Uppercase String: " + upperCaseString);

        // Convert string to lowercase
        String lowerCaseString = str.toLowerCase();
        System.out.println("Lowercase String: " + lowerCaseString);

        // Check if string starts with a specific prefix
        boolean startsWith = str.startsWith("Hello");
        System.out.println("String starts with 'Hello': " + startsWith);

        // Check if string ends with a specific suffix
        boolean endsWith = str.endsWith("World!");
        System.out.println("String ends with 'World!': " + endsWith);

        // Replace characters in a string
        String replacedString = str.replace('l', 'x');
        System.out.println("Replaced String: " + replacedString);

        // Split string into an array of substrings
        String[] splitString = str.split(", ");
        System.out.println("Split String: ");
        for (String s : splitString) {

```

```

        System.out.println(s);
    }

    // Trim leading and trailing whitespaces
    String str3 = "    Hello, World!    ";
    String trimmedString = str3.trim();
    System.out.println("Trimmed String: " + trimmedString);
}
}

```

## Event handling in Core Java

Event handling in Core Java involves three main components: the event source, the event object, and the event listener. The event source is the object on which the event occurs. The event object contains information about the event, such as its type and the state of the source object. The event listener is an object that receives the event and provides a response.

Here is an example of how to handle a button click event in Core Java:

```

import java.awt.*;
import java.awt.event.*;

public class ButtonClickExample extends Frame implements ActionListener {
    ButtonClickExample() {
        Button b = new Button("Click me!");
        b.setBounds(50, 100, 80, 30);
        add(b);
        b.addActionListener(this);
        setSize(300, 300);
        setLayout(null);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }

    public static void main(String[] args) {
        new ButtonClickExample();
    }
}

```

In this example, the `ButtonClickExample` class extends `Frame` and implements the `ActionListener` interface. The `ActionListener` interface requires the implementation of the `actionPerformed` method, which is called when an action event occurs. In this case, the action event is a button click. The

`actionPerformed` method prints a message to the console when the button is clicked.

The `ButtonClickExample` constructor creates a `Button` object and sets its properties. The `addActionListener` method is called on the button object and `this` is passed as an argument, indicating that the `ButtonClickExample` object is the event listener for the button click event. The `setSize`, `setLayout`, and `setVisible` methods are called to set the size, layout, and visibility of the frame.

The `main` method creates an instance of the `ButtonClickExample` class, which displays the frame and the button. When the button is clicked, the `actionPerformed` method is called and the message is printed to the console.

## Introduction to AWT in Core Java

AWT (Abstract Window Toolkit) is a set of APIs used by Java programmers to create graphical user interfaces (GUIs). It is part of the Java Foundation Classes (JFC) and provides a platform-independent way to develop rich, interactive user interfaces for desktop applications.

Here is an example of a simple AWT program that creates a window with a button:

```
import java.awt.*;
import java.awt.event.*;

public class AWTEExample extends Frame implements ActionListener {
    Button b;

    AWTEExample() {
        b = new Button("Click me");
        b.setBounds(30, 100, 80, 30);
        add(b);
        setSize(300, 300);
        setLayout(null);
        setVisible(true);
        b.addActionListener(this);
    }

    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }

    public static void main(String[] args) {
        new AWTEExample();
    }
}
```



This code creates a window with a button that, when clicked, prints “Button clicked!” to the console. The `AWTExample` class extends the `Frame` class, which represents a window in AWT. The `Button` class is used to create a button, and the `ActionListener` interface is implemented to handle button clicks.

### AWT controls

Here is an example of a simple Java program that uses AWT controls to create a GUI with a button and a label:

```
import java.awt.*;
import java.awt.event.*;

public class AWTExample extends Frame implements ActionListener {
    Label label;
    Button button;

    public AWTExample() {
        setLayout(new FlowLayout());

        label = new Label("Click the button!");
        add(label);

        button = new Button("Click me");
        add(button);
        button.addActionListener(this);

        setTitle("AWT Example");
        setSize(250, 100);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        label.setText("Button clicked!");
    }

    public static void main(String[] args) {
        new AWTExample();
    }
}
```

This code creates a window with a label and a button. When the button is clicked, the text of the label changes to “Button clicked!”.

### Layout managers in AWT

- AWT (Abstract Window Toolkit) is a Java package that provides a

platform-independent interface for creating Graphical User Interfaces (GUIs).

- Layout managers are used to arrange components in a container.
- AWT provides several layout managers, including `BorderLayout`, `FlowLayout`, `GridLayout`, `CardLayout`, and `GridBagLayout`.
- Each layout manager has its own rules for arranging components.
- `BorderLayout` arranges components in five regions: north, south, east, west, and center.
- `FlowLayout` arranges components in a row, with any extra space distributed between the components.
- `GridLayout` arranges components in a grid of rows and columns, with all components having the same size.
- `CardLayout` arranges components as a stack of cards, with only one card visible at a time.
- `GridBagLayout` is the most flexible layout manager, allowing components to be arranged in a grid with varying row heights and column widths.
- The `setLayout` method is used to set the layout manager for a container.
- The `add` method is used to add components to a container, with the layout manager determining the position and size of the component.
- The `pack` method is used to size the container to fit its components.

## Unit 2 - Web Page Designing

Here is an example of a simple HTML code for designing a web page:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My Web Page</title>
  </head>
  <body>
    <h1>Welcome to my web page</h1>
    <p>This is a paragraph of text on my web page.</p>
  </body>
</html>
```

This code creates a basic web page with a title, a heading, and a paragraph of text. The `<!DOCTYPE html>` declaration specifies that this document is an HTML5 document. The `<html>` element is the root element of an HTML page. The `<head>` element contains meta information about the document, such as the title. The `<title>` element specifies a title for the document, which is displayed in the browser's title bar or tab. The `<body>` element contains the visible page content, such as headings, paragraphs, images, hyperlinks, tables, lists, etc. The `<h1>` element defines a large heading, and the `<p>` element defines a paragraph.

## HTML in Web Page Designing

HTML stands for HyperText Markup Language. It is the standard markup language for creating web pages and other information that can be displayed in a web browser.

- HTML is used to structure content on the web and give meaning to the content.
- HTML consists of a series of elements, which are used to enclose, or wrap, different parts of the content to make it appear a certain way or act a certain way.
- The enclosing tags can make a word or image hyperlink to somewhere else, can italicize words, can make the font bigger or smaller, and so on.
- HTML is not a programming language, it is a markup language. A markup language is a set of markup tags, and HTML uses markup tags to describe web pages.
- HTML can embed programs written in a scripting language such as JavaScript, which affects the behavior and content of web pages.
- In summary, HTML is the foundation of web page design, providing the structure and meaning to the content on the web.

## List in Web Page Designing

There are two main types of lists used in web page designing: ordered lists and unordered lists. An ordered list is a numbered list, while an unordered list is a bulleted list.

Here is an example of an ordered list in HTML:

```
<ol>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ol>
```

And here is an example of an unordered list in HTML:

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
  <li>Item 3</li>
</ul>
```

## Table in Web Page Designing

1. Tables are used to organize and present data in a structured manner.
2. They are created using the `<table>` HTML tag, with rows defined by `<tr>` and cells defined by `<td>`.

3. Tables can have headers, defined by `<th>`, which are typically used to label columns or rows.
4. Tables can be styled using CSS to control their appearance, such as the width of columns, the alignment of text, and the color of cells.
5. Tables should be used for tabular data, not for layout purposes. Using tables for layout can result in accessibility issues and can make it difficult to maintain the page.
6. When designing tables, it is important to consider the readability of the data, and to use features such as alternating row colors or borders to improve the visual separation of data.
7. Tables can also be made responsive, allowing them to adapt to different screen sizes and devices.
8. It is important to ensure that tables are accessible to all users, including those using assistive technologies such as screen readers. This can be achieved by using semantic HTML and providing captions and summaries for tables.

### Images in Web Page Designing

1. **Images are an important aspect of web page designing** as they can convey information and emotions more effectively than text alone.
2. **Images can be used to enhance the visual appeal** of a web page and make it more engaging for the user.
3. **Images should be relevant to the content** of the web page and should be placed in a way that enhances the overall design and layout of the page.
4. **Images should be optimized for web use** to ensure that they load quickly and do not slow down the page. This can be done by compressing the images and using appropriate file formats.
5. **Images should be used in moderation** as too many images can clutter the page and make it difficult for the user to focus on the content.
6. **Images should be used in conjunction with text** to provide context and enhance the user's understanding of the content.
7. **Images should be accessible** to all users, including those with visual impairments. This can be achieved by providing alternative text for images and using appropriate contrast and color schemes.
8. **Images should be used ethically** and should not be used to mislead or deceive the user. Images should also be used in compliance with copyright laws.

### Frames in Web Page Designing

1. Frames are a way to divide a web page into multiple sections, each of which can display different content.
2. Frames are created using the `<frame>` and `<frameset>` HTML tags.
3. The `<frameset>` tag is used to define the layout of the frames on the page, while the `<frame>` tag is used to specify the content of each frame.

4. Frames can be useful for creating navigation menus, headers, and footers that remain visible while the user scrolls through the content of the page.
5. However, frames can also have some drawbacks, such as making it difficult for search engines to index the content of the page and for users to bookmark specific pages.
6. Due to these drawbacks, frames have largely been replaced by other techniques, such as CSS and JavaScript, for creating modern web page layouts.

### Forms in Web Page Designing

1. Forms are used to collect user input on a web page.
2. They can be used for various purposes such as user registration, login, feedback, and surveys.
3. Forms are created using HTML and can be styled using CSS.
4. The `<form>` tag is used to create a form in HTML.
5. Within the `<form>` tag, various form elements such as text fields, radio buttons, checkboxes, and dropdown menus can be added using the appropriate HTML tags.
6. The `name` attribute is used to specify the name of the form element, which is used to identify the data when the form is submitted.
7. The `action` attribute of the `<form>` tag specifies the URL to which the form data is sent when the form is submitted.
8. The `method` attribute specifies the HTTP method used to send the form data, either `GET` or `POST`.
9. Form validation can be performed using JavaScript to ensure that the user has entered valid data before the form is submitted.
10. Server-side validation can also be performed to ensure the data is valid and secure before being processed.

### CSS in Web Page Designing

CSS stands for Cascading Style Sheets. It is a stylesheet language used to describe the look and formatting of a document written in a markup language like HTML. CSS is used to enhance the appearance of web pages by defining colors, fonts, layouts, and more.

Some key points to remember when using CSS in web page designing are:

1. CSS allows for the separation of presentation and content, making it easier to maintain and update the design of a web page.
2. CSS can be used to create consistent styling across multiple pages of a website.
3. CSS can be used to create responsive designs that adapt to different screen sizes and devices.
4. CSS can be used to create animations and transitions to enhance the user experience.

5. CSS can be used to improve the accessibility of a web page by providing alternative styles for users with different needs.

Overall, CSS is a powerful tool for web page designing that allows for greater control over the appearance and behavior of a web page. It is an essential skill for any web designer to master.

### Document type definition in Web Page Designing

A Document Type Definition (DTD) is a set of markup declarations that define a document type for an SGML-family markup language (SGML, XML, HTML). A DTD defines the valid building blocks of an XML document. It sets the rules for the markup language, so that the structure of an XML document can be verified for correctness.

Here is an example of a DTD declaration in an XML document:

```
<!DOCTYPE note [  
<!ELEMENT note (to,from,heading,body)>  
<!ELEMENT to (#PCDATA)>  
<!ELEMENT from (#PCDATA)>  
<!ELEMENT heading (#PCDATA)>  
<!ELEMENT body (#PCDATA)>  

```

This DTD specifies that the `note` element must contain the elements `to`, `from`, `heading`, and `body` in that order. The `#PCDATA` keyword indicates that the elements can contain parsed character data.

In HTML, the `<!DOCTYPE>` declaration is used to specify the version of HTML that the page is written in. For example, the following declaration specifies that the page is written in HTML5:

```
<!DOCTYPE html>
```

This declaration must be the first line in the HTML document, before the `<html>` tag. It is not an HTML tag, but an instruction to the web browser about what version of HTML the page is written in. This helps the browser to render the page correctly.

### XML in Web Page Designing

XML (eXtensible Markup Language) is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. It is commonly used for the storage and transport of data on the web.

Here is an example of how XML can be used in web page designing:

```
<?xml version="1.0" encoding="UTF-8"?>  
<page>
```

```

<title>My Web Page</title>
<header>
  <logo src="logo.png" alt="My Logo" />
  <menu>
    <item href="index.html">Home</item>
    <item href="about.html">About</item>
    <item href="contact.html">Contact</item>
  </menu>
</header>
<content>
  <h1>Welcome to my web page!</h1>
  <p>This is some sample content.</p>
</content>
<footer>
  <p>Copyright © 2023 My Web Page</p>
</footer>
</page>

```

This XML document describes the structure and content of a simple web page. It can be transformed into HTML using XSLT (eXtensible Stylesheet Language Transformations) or other methods to be displayed in a web browser.

## DTD in Web Page Designing

A Document Type Definition (DTD) is a set of markup declarations that define a document type for an SGML-family markup language (SGML, XML, HTML). A DTD defines the valid building blocks of an XML document. It sets the rules for the markup language, so that the structure of the document can be verified.

Here is an example of a simple DTD for an XML document that might be used for a list of people:

```

<!DOCTYPE people [
  <!ELEMENT people (person*)>
  <!ELEMENT person (name, email)>
  <!ELEMENT name (#PCDATA)>
  <!ELEMENT email (#PCDATA)>
]>

```

This DTD specifies that the **people** element contains zero or more **person** elements, and that each **person** element must contain a **name** element followed by an **email** element. The **name** and **email** elements can contain parsed character data (PCDATA), which means that they can contain any text.

In web page designing, a DTD is used to define the structure and content of an HTML or XHTML document. By specifying a DTD, the web designer can ensure that the web page will be displayed correctly by web browsers that support the specified DTD. It is important to include a DTD in a web page to ensure

that the page is displayed consistently across different web browsers.

### XML schemes in Web Page Designing

XML (eXtensible Markup Language) is a markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable. XML is commonly used in web page designing to store and transport data.

Here is an example of how XML can be used in web page designing:

```
<?xml version="1.0" encoding="UTF-8"?>
<page>
  <title>My Web Page</title>
  <header>
    <logo src="logo.png" alt="My Logo" />
    <menu>
      <item href="index.html">Home</item>
      <item href="about.html">About</item>
      <item href="contact.html">Contact</item>
    </menu>
  </header>
  <content>
    <h1>Welcome to my web page!</h1>
    <p>This is some sample content.</p>
  </content>
  <footer>
    <p>Copyright © 2023 My Web Page</p>
  </footer>
</page>
```

This XML document defines the structure and content of a web page. The `<page>` element is the root element of the document, and it contains child elements such as `<title>`, `<header>`, `<content>`, and `<footer>` that define the different sections of the web page. The `<menu>` element contains a list of `<item>` elements that represent the navigation menu of the web page.

Using XML in web page designing allows for a clear separation of content and presentation, making it easier to maintain and update the web page. Additionally, XML data can be easily accessed and manipulated using various programming languages and tools.

### Object Models in Web Page Designing

Object models are used in web page designing to represent the structure and behavior of the objects within a web page. An object model is a conceptual representation of the objects and their relationships within a system. In web



page designing, object models are used to define the structure of the web page and the interactions between the different elements on the page.

Here is an example of how an object model can be used in web page designing:

```
// Define a "Page" object
function Page(title, content) {
    this.title = title;
    this.content = content;
}

// Define a "Header" object
function Header(title) {
    this.title = title;
}

// Define a "Footer" object
function Footer(content) {
    this.content = content;
}

// Create a new "Page" object
var myPage = new Page("My Web Page", "Welcome to my web page!");

// Create a new "Header" object
var myHeader = new Header("My Web Page");

// Create a new "Footer" object
var myFooter = new Footer("Copyright 2023");

// Add the "Header" and "Footer" objects to the "Page" object
myPage.header = myHeader;
myPage.footer = myFooter;
```

In this example, we have defined three objects: **Page**, **Header**, and **Footer**. These objects represent the different elements of a web page. We have also created instances of these objects and added them to the **myPage** object to represent the structure of the web page.

This is just one way that object models can be used in web page designing. There are many other ways that object models can be used to represent the structure and behavior of web pages.

## Presenting and using XML in Web Page Designing

XML (eXtensible Markup Language) is a markup language that is used to store and transport data. It is a flexible format that can be used to create structured documents and data sets. XML can be used in web page designing to present

data in a structured and organized manner.

Here is an example of how XML can be used in web page designing:

```
<?xml version="1.0" encoding="UTF-8"?>
<catalog>
  <book id="bk101">
    <author>Gambardella, Matthew</author>
    <title>XML Developer's Guide</title>
    <genre>Computer</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description>An in-depth look at creating applications with XML.</description>
  </book>
  <book id="bk102">
    <author>Ralls, Kim</author>
    <title>Midnight Rain</title>
    <genre>Fantasy</genre>
    <price>5.95</price>
    <publish_date>2000-12-16</publish_date>
    <description>A former architect battles corporate zombies, an evil sorceress, and her
  </book>
</catalog>
```

This XML document contains a catalog of books, with each book having its own set of elements such as author, title, genre, price, publish\_date, and description. This data can be presented on a web page using various methods such as XSLT (eXtensible Stylesheet Language Transformations) or by using JavaScript to parse the XML data and generate HTML content.

Here is an example of how the above XML data can be presented on a web page using XSLT:

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:template match="/">
  <html>
  <body>
    <h2>Book Catalog</h2>
    <table border="1">
      <tr bgcolor="#9acd32">
        <th>Title</th>
        <th>Author</th>
      </tr>
      <xsl:for-each select="catalog/book">
        <tr>
          <td><xsl:value-of select="title"/></td>
          <td><xsl:value-of select="author"/></td>
```

```

        </tr>
      </xsl:for-each>
    </table>
  </body>
</html>
</xsl:template>
</xsl:stylesheet>

```

This XSLT stylesheet transforms the XML data into an HTML table, presenting the title and author of each book in the catalog. The resulting HTML can be embedded into a web page to present the data to the user.

In summary, XML can be a useful tool in web page designing for presenting and organizing data in a structured manner. It can be used in conjunction with other technologies such as XSLT and JavaScript to generate dynamic and interactive web pages.

## Using XML Processors in Web Page Designing

XML processors are used in web page designing to parse and manipulate XML data. Here is an example of how to use an XML processor in JavaScript to parse an XML string and extract information from it:

```

let xmlString = `<?xml version="1.0" encoding="UTF-8"?>
<catalog>
  <book id="bk101">
    <author>Gambardella, Matthew</author>
    <title>XML Developer's Guide</title>
    <genre>Computer</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description>An in-depth look at creating applications with XML.</description>
  </book>
</catalog>`;

let parser = new DOMParser();
let xmlDoc = parser.parseFromString(xmlString, "text/xml");

let book = xmlDoc.getElementsByTagName("book")[0];
let title = book.getElementsByTagName("title")[0].childNodes[0].nodeValue;

console.log(title); // Output: "XML Developer's Guide"

```

This code uses the `DOMParser` object to parse the XML string and create an XML document object. The `getElementsByTagName` method is then used to access the `book` and `title` elements, and the `nodeValue` property is used to extract the text content of the `title` element. This information can then be used in the web page design to display the desired content to the user.

## DOM and SAX in Web Page Designing

- **DOM (Document Object Model)** is a platform and language-neutral interface that allows programs and scripts to dynamically access and update the content, structure, and style of a document.
- DOM represents a document as a tree structure, where each node in the tree represents a part of the document, such as an element, attribute, or text.
- DOM provides a standard set of methods for accessing and manipulating the nodes in the tree, allowing developers to create dynamic web pages that can update their content in response to user actions.
- **SAX (Simple API for XML)** is an event-driven, serial-access mechanism for accessing XML documents.
- SAX provides a standard interface for parsing XML documents, allowing developers to write programs that can read and process XML data in a streaming fashion.
- Unlike DOM, which loads the entire document into memory and allows random access to its nodes, SAX reads the document sequentially and generates events for the elements, attributes, and text it encounters.
- SAX is often used for processing large XML documents or for situations where memory usage is a concern, as it does not require the entire document to be loaded into memory.
- Both DOM and SAX have their advantages and disadvantages, and the choice between them depends on the specific needs of the application. DOM is generally easier to use and provides more flexibility, while SAX is more efficient for processing large documents or for applications with limited memory.

## Dynamic HTML in Web Page Designing

Dynamic HTML, or DHTML, is a collection of technologies used together to create interactive and animated web sites by using a combination of a static markup language (such as HTML), a client-side scripting language (such as JavaScript), a presentation definition language (such as CSS), and the Document Object Model (DOM).

Here is an example of a simple DHTML code that changes the text color of a paragraph when the user clicks on a button:

```
<!DOCTYPE html>
<html>
<head>
<style>
p {
  color: black;
}
</style>
```

```

</head>
<body>

<p id="myP">This is a paragraph.</p>

<button onclick="myFunction()">Click me</button>

<script>
function myFunction() {
    document.getElementById("myP").style.color = "red";
}
</script>

</body>
</html>

```

This code creates a paragraph with the text “This is a paragraph.” and a button with the text “Click me”. When the user clicks on the button, the JavaScript function `myFunction()` is called, which changes the color of the text in the paragraph to red by modifying the `style` property of the `p` element.

## Unit 3 - Scripting

Here is an example of a simple script in Python:

```

# This is a comment
# Print "Hello, World!" to the console
print("Hello, World!")

```

This script uses the `print()` function to output the string "Hello, World!" to the console. The first two lines are comments, which are ignored by the Python interpreter and are used to provide additional information about the code.

### Java script in Scripting

Here is an example of a simple JavaScript code that can be used in scripting:

```

// This is a comment
// This script will print "Hello, World!" to the console

console.log("Hello, World!");

```

### Introduction to JavaScript

JavaScript is a high-level, interpreted programming language that is commonly used to add interactivity to web pages. It is a versatile language that can be used for a wide range of tasks, including creating dynamic user interfaces, handling user input, and manipulating data.

Here is an example of a simple JavaScript program that displays a message on a web page:

```
document.write('Hello, World!');
```

This code uses the `document.write()` method to output the string 'Hello, World!' to the web page. When the page is loaded, the message will be displayed to the user.

JavaScript is a powerful language that can be used to create complex web applications. It is an essential tool for any web developer and is widely used in the industry. If you are interested in learning more about JavaScript, there are many resources available online to help you get started.

## Documents in JavaScript

Here is an example of how to create, access, and modify a document in JavaScript:

```
// Create a new document
let doc = document.implementation.createHTMLDocument("New Document");

// Access the document's body
let body = doc.body;

// Create a new element
let p = doc.createElement("p");

// Set the element's text content
p.textContent = "This is some text.";

// Append the element to the body
body.appendChild(p);

// Modify the element's text content
p.textContent = "This is some updated text.";
```

## Forms in JavaScript

Here is an example of a simple form in JavaScript:

```
// Get the form element
var form = document.getElementById('myForm');

// Add an event listener to the form
form.addEventListener('submit', function(event) {
    // Prevent the default form submission behavior
    event.preventDefault();
});
```

```

    // Get the form data
    var formData = new FormData(form);

    // Log the form data to the console
    for (var pair of formData.entries()) {
        console.log(pair[0] + ': ' + pair[1]);
    }
});

```

#### Code for statements in JavaScript:

```

// if statement
if (condition) {
    // code to be executed if condition is true
}

// if-else statement
if (condition) {
    // code to be executed if condition is true
} else {
    // code to be executed if condition is false
}

// if-else-if statement
if (condition1) {
    // code to be executed if condition1 is true
} else if (condition2) {
    // code to be executed if condition2 is true
} else {
    // code to be executed if neither condition1 nor condition2 is true
}

// switch statement
switch(expression) {
    case value1:
        // code to be executed if expression === value1
        break;
    case value2:
        // code to be executed if expression === value2
        break;
    ...
    default:
        // code to be executed if expression doesn't match any case
}

```

#### Functions in JavaScript

A function in JavaScript is a block of code designed to perform a particular task. A function is executed when it is called or invoked. Here is an example of a function in JavaScript:

```
function greet(name) {  
  console.log("Hello, " + name);  
}  
  
greet("John"); // Output: Hello, John
```

In this example, we have defined a function named **greet** that takes one parameter, **name**. When we call the function and pass in a value for the **name** parameter, the function will execute the code within its block and log a greeting message to the console.

Functions can also return values. Here is an example of a function that returns the sum of two numbers:

```
function add(a, b) {  
  return a + b;  
}  
  
let result = add(1, 2); // result = 3
```

In this example, the **add** function takes two parameters, **a** and **b**, and returns their sum. When we call the function and assign its return value to a variable, we can use that value in our code.

Functions are a powerful feature of JavaScript that allow us to write reusable and modular code. They can take any number of parameters and can return any type of value.

## Objects in JavaScript

An object in JavaScript is a collection of properties, where each property has a name and a value. Properties can be primitive values, other objects, or functions. Here is an example of creating an object in JavaScript:

```
let person = {  
  firstName: "John",  
  lastName: "Doe",  
  age: 25,  
  greet: function() {  
    console.log("Hello, my name is " + this.firstName + " " + this.lastName);  
  }  
};
```

In this example, we create an object called **person** with four properties: **firstName**, **lastName**, **age**, and **greet**. The **greet** property is a function that



logs a greeting to the console using the `firstName` and `lastName` properties of the `person` object.

You can access the properties of an object using dot notation or bracket notation. Here is an example of accessing the `firstName` property of the `person` object using both notations:

```
console.log(person.firstName); // Output: John
console.log(person["firstName"]); // Output: John
```

You can also add new properties to an object or modify existing properties. Here is an example of adding a new property called `email` to the `person` object and modifying the `age` property:

```
person.email = "john.doe@example.com";
person["age"] = 26;
```

After adding the `email` property and modifying the `age` property, the `person` object now looks like this:

```
{
  firstName: "John",
  lastName: "Doe",
  age: 26,
  email: "john.doe@example.com",
  greet: function() {
    console.log("Hello, my name is " + this.firstName + " " + this.lastName);
  }
}
```

## Introduction to AJAX in JavaScript

AJAX stands for Asynchronous JavaScript and XML. It is a technique for creating fast and dynamic web pages without reloading the entire page. AJAX allows web pages to send and receive data from a server asynchronously, in the background, without interfering with the display and behavior of the existing page.

Here is an example of how to use AJAX in JavaScript to send a request to a server and receive a response:

```
// Create a new XMLHttpRequest object
var xhr = new XMLHttpRequest();

// Open a new connection, using the GET request on the URL endpoint
xhr.open('GET', 'https://api.example.com/data', true);

// Set the request header
xhr.setRequestHeader('Content-Type', 'application/json');
```

```

// Send the request
xhr.send();

// This will be called after the response is received
xhr.onload = function() {
  if (xhr.status != 200) {
    // analyze HTTP response
    alert(`Error ${xhr.status}: ${xhr.statusText}`);
  } else {
    // parse JSON data
    let data = JSON.parse(xhr.response);
    // process the data
    // ...
  }
};

xhr.onerror = function() {
  alert("Request failed");
};

```

## Networking in Scripting

Here is an example of a Python script that uses the `socket` module to establish a network connection and send data:

```

import socket

# create a socket object
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# get local machine name
host = socket.gethostname()

# connection to hostname on the port.
s.connect((host, 8080))

# send a thank you message to the client.
s.sendall(b'Thank you for connecting')

# receive data from the client
data = s.recv(1024)

# close the socket
s.close()

# print the received data

```

```
print(data.decode('utf-8'))
```

## Internet Addressing in Networking

An Internet address is a unique identifier that is assigned to a device connected to the Internet. It is used to route data to and from the device. The most common form of Internet addressing is the Internet Protocol (IP) address. An IP address is a numerical label assigned to each device participating in a computer network that uses the Internet Protocol for communication. There are two versions of the IP address: IPv4 and IPv6. IPv4 addresses are 32-bit numbers, while IPv6 addresses are 128-bit numbers. Both versions of the IP address are used to identify devices on the Internet and to route data to and from those devices.

Here is an example of how to assign an IPv4 address to a device in Python:

```
import socket

# create a socket object
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# get local machine name
host = socket.gethostname()

# bind the socket to a public host, and a well-known port
s.bind((host, 80))

# become a server socket
s.listen(5)
```

## InetAddress in Networking

InetAddress is a class in the `java.net` package that represents an Internet Protocol (IP) address. Here is an example of how to use the InetAddress class to get the IP address of a given hostname:

```
import java.net.InetAddress;
import java.net.UnknownHostException;

public class InetAddressExample {
    public static void main(String[] args) {
        try {
            InetAddress address = InetAddress.getByName("www.example.com");
            System.out.println(address.getHostAddress());
        } catch (UnknownHostException e) {
            e.printStackTrace();
        }
    }
}
```

```

    }
}

```

## Factory Methods in Networking

Factory methods are a design pattern commonly used in object-oriented programming. They provide a way to create objects without specifying the exact class of object that will be created. This can be useful in networking, where the specific implementation of a network connection may vary depending on the underlying protocol or platform.

Here is an example of a factory method in Python that creates a network connection object:

```

class NetworkConnectionFactory:
    @staticmethod
    def create_connection(protocol: str):
        if protocol == 'TCP':
            return TCPConnection()
        elif protocol == 'UDP':
            return UDPConnection()
        else:
            raise ValueError(f'Unknown protocol: {protocol}')

```

In this example, the `create_connection` method takes a `protocol` argument that specifies the type of connection to create. Depending on the value of this argument, the method returns an instance of either the `TCPConnection` or `UDPConnection` class. This allows the caller to create a network connection without knowing the specific class of the connection object.

## Instance Methods in Networking

Here is an example of instance methods in networking using Python:

```

import socket

class Network:
    def __init__(self, host, port):
        self.host = host
        self.port = port
        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    def connect(self):
        self.socket.connect((self.host, self.port))

    def send(self, data):
        self.socket.sendall(data.encode())

```

```

def receive(self, buffer_size):
    return self.socket.recv(buffer_size).decode()

def close(self):
    self.socket.close()

```

## TCP/IP Client Sockets in Networking

Here is an example of a simple TCP/IP client socket written in Python:

```

import socket

# create a socket object
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# get local machine name
host = socket.gethostname()

# connection to hostname on the port.
s.connect((host, 9999))

# Receive no more than 1024 bytes
msg = s.recv(1024)

s.close()

print(msg.decode('ascii'))

```

## URL in Networking

A URL (Uniform Resource Locator) is a reference to a web resource that specifies its location on a computer network and a mechanism for retrieving it. A URL is a specific type of Uniform Resource Identifier (URI), although many people use the two terms interchangeably. URLs occur most commonly to reference web pages (http), but are also used for file transfer (ftp), email (mailto), database access (JDBC), and many other applications.

Here is an example of a URL in Python:

```

import urllib.request

url = 'http://www.example.com'
response = urllib.request.urlopen(url)
webContent = response.read()

print(webContent[0:300])

```

## URL Connection in Networking

A URL connection is a connection between a Java application and a URL. It can be used to read from or write to a resource specified by the URL. Here is an example of how to use a URL connection to read the contents of a web page:

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.URL;
import java.net.URLConnection;

public class URLConnectionExample {
    public static void main(String[] args) {
        try {
            URL url = new URL("http://www.example.com");
            URLConnection connection = url.openConnection();
            BufferedReader in = new BufferedReader(new InputStreamReader(connection.getInputStream()));
            String inputLine;
            while ((inputLine = in.readLine()) != null) {
                System.out.println(inputLine);
            }
            in.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

## TCP/IP Server Sockets in Networking

Here is an example of a simple TCP/IP server socket in Python:

```
import socket

# create a socket object
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# get local machine name
host = socket.gethostname()

port = 9999

# bind the socket to a public host, and a well-known port
server_socket.bind((host, port))

# become a server socket
server_socket.listen(5)
```

```

while True:
    # establish a connection
    client_socket, addr = server_socket.accept()

    print("Got a connection from %s" % str(addr))

    msg = 'Thank you for connecting'+ "\r\n"
    client_socket.send(msg.encode('ascii'))
    client_socket.close()

```

### Datagram in Networking

A datagram is a self-contained, independent entity of data carrying sufficient information to be routed from the source to the destination computer without reliance on earlier exchanges between this source and destination computer and the transporting network. Here is an example of a simple Python program that sends a UDP datagram:

```

import socket

UDP_IP = "127.0.0.1"
UDP_PORT = 5005
MESSAGE = "Hello, World!"

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.sendto(MESSAGE.encode(), (UDP_IP, UDP_PORT))

```

### Unit 4 - Enterprise Java Bean

Here is an example of an Enterprise Java Bean (EJB) code:

```

import javax.ejb.Stateless;

@Stateless
public class ExampleBean {

    public String sayHello(String name) {
        return "Hello, " + name + "!";
    }
}

```

This is a simple example of a stateless session bean that has a method **sayHello** which takes a **String** parameter **name** and returns a greeting message. The **@Stateless** annotation indicates that this bean is a stateless session bean.

## Preparing a Class to be a JavaBeans

A JavaBean is a reusable software component that follows certain design conventions. To prepare a class to be a JavaBean, it must meet the following requirements:

1. The class must implement the `Serializable` interface.
2. The class must have a public no-argument constructor.
3. The class must have properties that are accessed through getter and setter methods that follow the naming convention of `getPropertyName` and `setPropertyName`.
4. The class may have an event notification mechanism through the use of listeners and event objects.

Here is an example of a simple JavaBean class:

```
import java.io.Serializable;

public class MyBean implements Serializable {
    private String name;
    private int age;

    public MyBean() {
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

## Creating a JavaBeans

A JavaBean is a reusable software component that follows certain design conventions. Here is an example of how to create a simple JavaBean:



```

public class MyBean implements java.io.Serializable {
    private String property1;
    private int property2;

    public MyBean() {
    }

    public String getProperty1() {
        return property1;
    }

    public void setProperty1(String property1) {
        this.property1 = property1;
    }

    public int getProperty2() {
        return property2;
    }

    public void setProperty2(int property2) {
        this.property2 = property2;
    }
}

```

### JavaBeans Properties

JavaBeans properties are accessed through getter and setter methods. Here is an example of a simple Java class with a property called `name`:

```

public class Person {
    private String name;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }
}

```

The `getName` method is the getter for the `name` property, and the `setName` method is the setter. These methods allow the `name` property to be read and modified. The `name` property itself is private, so it can only be accessed through these methods. This is a common pattern in JavaBeans.

## Types of beans in Enterprise Java Bean

Enterprise Java Beans (EJB) is a server-side component architecture for the Java Platform, Enterprise Edition (Java EE). There are three types of beans in EJB:

1. **Session Beans:** These beans represent the business logic of an application and can be either stateful or stateless. Stateful session beans maintain state across multiple method invocations, while stateless session beans do not.
2. **Entity Beans:** These beans represent persistent data and are used to manage the interactions between the application and the database. Entity beans can be either container-managed or bean-managed.
3. **Message-Driven Beans:** These beans are used to process messages asynchronously. They act as a listener for a particular messaging type, such as Java Message Service (JMS), and perform some action when a message is received.

Each type of bean has its own specific use case and can be used in combination to build robust and scalable enterprise applications.

## Stateful Session bean in Enterprise Java Bean

A stateful session bean is a type of enterprise bean that maintains conversational state with the client. Here is an example of a stateful session bean:

```
import javax.ejb.Stateful;

@Stateful
public class ExampleStatefulBean {
    private int counter = 0;

    public void incrementCounter() {
        counter++;
    }

    public int getCounter() {
        return counter;
    }
}
```

This bean maintains a counter that can be incremented and retrieved by the client. The `@Stateful` annotation indicates that this is a stateful session bean. The state of the bean is maintained across multiple method invocations by the same client.

## Stateless Session bean in Enterprise Java Bean

A Stateless Session Bean is a type of Enterprise Java Bean (EJB) that does not maintain conversational state with the client. Here is an example of a Stateless Session Bean:

```
import javax.ejb.Stateless;

@Stateless
public class MyStatelessBean implements MyStatelessBeanRemote {
    public MyStatelessBean() {}

    public String myMethod() {
        // business logic here
        return "result";
    }
}
```

This bean is annotated with `@Stateless` to indicate that it is a Stateless Session Bean. The business logic is implemented in the `myMethod()` method. This bean can be accessed remotely through the `MyStatelessBeanRemote` interface.

### Entity bean in Enterprise Java Bean

An entity bean represents a business object in a persistent storage mechanism. Here is an example of an entity bean that represents a bank account:

```
import javax.persistence.*;

@Entity
public class BankAccount {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String accountNumber;
    private double balance;

    public BankAccount() {}

    public BankAccount(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public Long getId() {
        return id;
    }

    public String getAccountNumber() {
```

```

        return accountNumber;
    }

    public void setAccountNumber(String accountNumber) {
        this.accountNumber = accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }
}

```

This entity bean uses annotations to specify the mapping between the class and the database table. The `@Entity` annotation specifies that this class is an entity bean. The `@Id` and `@GeneratedValue` annotations specify that the `id` field is the primary key and that its value is automatically generated. The other fields represent the columns in the database table. The getters and setters provide access to the entity's state.

## Java Database Connectivity (JDBC)

Java Database Connectivity (JDBC) is an application programming interface (API) for the programming language Java, which defines how a client may access a database. Here is an example of how to connect to a database using JDBC:

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class JdbcExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String username = "myusername";
        String password = "mypassword";

        System.out.println("Connecting to database...");

        try (Connection connection = DriverManager.getConnection(url, username, password)) {
            System.out.println("Database connected!");
        } catch (SQLException e) {
            throw new IllegalStateException("Cannot connect to the database!", e);
        }
    }
}

```

```
}
```

This code connects to a MySQL database running on the local machine on port 3306. The database name is `mydatabase`, the username is `myusername`, and the password is `mypassword`. The `try-with-resources` statement is used to automatically close the `Connection` object when it is no longer needed.

## Merging Data from Multiple Tables in JDBC

Here is an example of how you can merge data from multiple tables in JDBC:

```
import java.sql.*;

public class MergeTables {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/databaseName";
        String user = "username";
        String password = "password";

        try (Connection conn = DriverManager.getConnection(url, user, password);
            Statement stmt = conn.createStatement()) {

            String query = "SELECT t1.column1, t2.column2 FROM table1 t1 INNER JOIN table2 t2 ON t1.id = t2.id";
            ResultSet rs = stmt.executeQuery(query);

            while (rs.next()) {
                String column1 = rs.getString("column1");
                String column2 = rs.getString("column2");
                System.out.println(column1 + " " + column2);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

This code connects to a MySQL database and uses an `INNER JOIN` to merge data from two tables, `table1` and `table2`, based on a common column, `id`. The resulting data is then printed to the console. You can modify the query and the code to suit your specific needs.

## Joining in JDBC

Joining in JDBC can be done by executing a SQL statement that includes a `JOIN` clause. Here is an example of how to perform a `JOIN` operation between two tables in JDBC:

```
// Assume conn is an active Connection
```

```
Statement stmt = conn.createStatement();
String query = "SELECT * FROM table1 INNER JOIN table2 ON table1.id = table2.id";
ResultSet rs = stmt.executeQuery(query);

while (rs.next()) {
    // Process the row
}
```

This code creates a `Statement` object and executes a SQL query that performs an `INNER JOIN` between `table1` and `table2` on the `id` column. The results of the query are stored in a `ResultSet` object, which can be iterated to process each row of the result.

## Manipulating in JDBC

Here is an example of how to manipulate data in a database using JDBC:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.Statement;

public class JdbcExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "username";
        String password = "password";

        try (Connection conn = DriverManager.getConnection(url, user, password)) {
            Statement stmt = conn.createStatement();
            String sql = "INSERT INTO mytable (column1, column2) VALUES ('value1', 'value2')";
            stmt.executeUpdate(sql);
            System.out.println("Data inserted successfully.");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

This code connects to a MySQL database, creates a `Statement` object, and executes an `INSERT` statement to add a new row to the `mytable` table. The `executeUpdate` method is used to execute statements that manipulate data in the database, such as `INSERT`, `UPDATE`, and `DELETE` statements.

## Databases with JDBC in JDBC

Here is an example of how to connect to a database using JDBC:

```
import java.sql.Connection;
```

```

import java.sql.DriverManager;
import java.sql.SQLException;

public class DatabaseConnection {
    public static void main(String[] args) {
        // Replace with your database URL, username, and password
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String username = "myusername";
        String password = "mypassword";

        // Connect to the database
        try (Connection connection = DriverManager.getConnection(url, username, password)) {
            System.out.println("Connected to the database");
        } catch (SQLException e) {
            System.out.println("Error connecting to the database");
            e.printStackTrace();
        }
    }
}

```

This code connects to a MySQL database using the JDBC driver. The `url` variable should be updated with the URL of your database, and the `username` and `password` variables should be updated with your database username and password. The `try-with-resources` statement is used to automatically close the `Connection` object when it is no longer needed.

## Prepared Statements in JDBC

A `PreparedStatement` is a precompiled SQL statement that can be executed multiple times without having to be recompiled for each execution. This can improve the performance of database operations. Here is an example of how to use a `PreparedStatement` in JDBC:

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class PreparedStatementExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "username";
        String password = "password";
        String query = "SELECT * FROM users WHERE age > ? AND city = ?";

        try (Connection conn = DriverManager.getConnection(url, user, password);

```

```

        PreparedStatement pstmt = conn.prepareStatement(query)) {

    pstmt.setInt(1, 30); // Set the first parameter to 30
    pstmt.setString(2, "New York"); // Set the second parameter to "New York"

    ResultSet rs = pstmt.executeQuery();

    while (rs.next()) {
        System.out.println(rs.getString("name") + ", " + rs.getInt("age") + ", " + rs.getString("city"));
    }
} catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

### Transaction Processing in JDBC

```

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class TransactionProcessing {
    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        try {
            // Step 1: Register JDBC driver
            Class.forName("com.mysql.jdbc.Driver");

            // Step 2: Open a connection
            conn = DriverManager.getConnection("jdbc:mysql://localhost/EMP", "username", "password");

            // Step 3: Disable auto-commit mode
            conn.setAutoCommit(false);

            // Step 4: Create a statement object
            stmt = conn.createStatement();

            // Step 5: Execute a series of SQL statements
            String sql1 = "INSERT INTO Employees VALUES (106, 20, 'Rita', 'Tez')";
            stmt.executeUpdate(sql1);

            String sql2 = "UPDATE Employees SET age=30 WHERE id=106";
            stmt.executeUpdate(sql2);
        } catch (Exception e) {
            e.printStackTrace();
        } finally {
            if (stmt != null) {
                stmt.close();
            }
            if (conn != null) {
                conn.close();
            }
        }
    }
}

```



```

        // Step 6: Commit the transaction
        conn.commit();

        System.out.println("Transaction committed successfully.");
    } catch (SQLException se) {
        // Handle errors for JDBC
        try {
            // Rollback the transaction in case of errors
            if (conn != null) {
                conn.rollback();
            }
            System.out.println("Transaction rolled back.");
        } catch (SQLException se2) {
            se2.printStackTrace();
        }
        se.printStackTrace();
    } catch (Exception e) {
        // Handle errors for Class.forName
        e.printStackTrace();
    } finally {
        // Step 7: Clean-up environment
        try {
            if (stmt != null) {
                stmt.close();
            }
        } catch (SQLException se2) {
            se2.printStackTrace();
        }
        try {
            if (conn != null) {
                conn.close();
            }
        } catch (SQLException se) {
            se.printStackTrace();
        }
    }
}
}
}

```

### Stored Procedures in JDBC

A stored procedure is a precompiled set of SQL statements that are stored in a database. They can be called from a Java program using JDBC. Here is an example of how to call a stored procedure using JDBC:

```

import java.sql.*;

public class StoredProcedureExample {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydatabase";
        String user = "username";
        String password = "password";

        try (Connection conn = DriverManager.getConnection(url, user, password);
            CallableStatement stmt = conn.prepareCall("{call my_stored_procedure(?, ?)}"))

            // Set input parameters
            stmt.setInt(1, 123);
            stmt.setString(2, "example");

            // Register output parameters
            stmt.registerOutParameter(3, Types.INTEGER);
            stmt.registerOutParameter(4, Types.VARCHAR);

            // Execute stored procedure
            stmt.execute();

            // Get output parameters
            int output1 = stmt.getInt(3);
            String output2 = stmt.getString(4);

            // Process results
            System.out.println("Output 1: " + output1);
            System.out.println("Output 2: " + output2);
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}

```

This code calls a stored procedure named `my_stored_procedure` with two input parameters and two output parameters. The input parameters are set using the `setInt` and `setString` methods, and the output parameters are registered using the `registerOutParameter` method. The stored procedure is executed using the `execute` method, and the output parameters are retrieved using the `getInt` and `getString` methods. The results are then processed as needed.

## Unit 5 - Servlets

A servlet is a Java program that runs on a web server and handles HTTP requests and responses. Here is an example of a simple servlet that responds to

a GET request with a “Hello, World!” message:

```
import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class HelloWorldServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        response.getWriter().println("<h1>Hello, World!</h1>");
    }
}
```

This servlet extends the `HttpServlet` class, which provides methods for handling HTTP requests. The `doGet` method is called when the servlet receives a GET request. In this example, the servlet sets the content type of the response to “text/html” and writes a “Hello, World!” message to the response using the `getWriter` method of the `HttpServletResponse` object.

## Servlet Overview and Architecture in Servlets

A servlet is a Java program that runs on a web server and is used to handle HTTP requests and generate responses. Servlets are managed by a servlet container, which is a component of a web server or application server that provides the runtime environment for servlets.

The servlet container is responsible for managing the lifecycle of servlets, including initializing, invoking, and destroying them. It also handles the communication between the servlets and the web server, and provides services such as request dispatching, session management, and security.

The architecture of a servlet-based application typically consists of a web server, a servlet container, and one or more servlets. The web server receives HTTP requests from clients and forwards them to the servlet container. The servlet container then invokes the appropriate servlet to handle the request and generate a response, which is sent back to the client via the web server.

Servlets can be used to handle a wide range of tasks, including generating dynamic content, processing form data, and managing user sessions. They provide a powerful and flexible way to build web applications, and are widely used in Java-based web development.

## Interface Servlet and the Servlet Life Cycle in Servlets

The `javax.servlet.Servlet` interface defines the methods that all servlets must implement. A servlet class must implement this inter-

face either directly or by extending a class that implements it, such as `javax.servlet.http.HttpServlet`.

The servlet life cycle consists of the following phases:

1. **Servlet instance creation:** The servlet container creates an instance of the servlet class when it is first requested by a client.
2. **Initialization:** The servlet container calls the `init` method of the servlet to initialize it. This method is called only once during the life cycle of the servlet.
3. **Request handling:** The servlet container calls the `service` method of the servlet to handle client requests. This method is called once for each request received by the servlet.
4. **Removal from service:** The servlet container calls the `destroy` method of the servlet to remove it from service. This method is called only once during the life cycle of the servlet, when the servlet is being removed from service.

Here is an example of a simple servlet that implements the `Servlet` interface and overrides the `init`, `service`, and `destroy` methods:

```
import java.io.IOException;
import javax.servlet.Servlet;
import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;

public class MyServlet implements Servlet {
    private ServletConfig config;

    @Override
    public void init(ServletConfig config) throws ServletException {
        this.config = config;
        System.out.println("Servlet initialized");
    }

    @Override
    public void service(ServletRequest req, ServletResponse res) throws ServletException, IOException {
        System.out.println("Handling request");
    }

    @Override
    public void destroy() {
        System.out.println("Servlet destroyed");
    }
}
```

```

@Override
public ServletConfig getServletConfig() {
    return config;
}

@Override
public String getServletInfo() {
    return "MyServlet";
}
}

```

### Handling HTTP get Requests in Servlets

To handle HTTP GET requests in a servlet, you need to override the `doGet` method of the `HttpServlet` class. Here is an example:

```

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class MyServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        // your code here
    }
}

```

In the `doGet` method, you can access the request parameters using the `request` object and generate a response using the `response` object. For example, to get the value of a request parameter named `name`, you can use the following code:

```
String name = request.getParameter("name");
```

To send a response to the client, you can use the `response` object's `getWriter` method to get a `PrintWriter` object, and then use its `println` method to write the response. For example, to send a plain text response to the client, you can use the following code:

```

response.setContentType("text/plain");
PrintWriter out = response.getWriter();
out.println("Hello, " + name);

```

### Handling HTTP post Requests in Servlets

To handle HTTP POST requests in a servlet, you need to override the `doPost` method of the `HttpServlet` class. Here is an example of how to do this:

```

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class MyServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException {
        // Get the request parameters
        String param1 = request.getParameter("param1");
        String param2 = request.getParameter("param2");

        // Process the request
        // ...

        // Set the response content type
        response.setContentType("text/html");

        // Write the response
        response.getWriter().println("POST request processed");
    }
}

```

In the above example, the `doPost` method is overridden to handle HTTP POST requests. The request parameters are retrieved using the `getParameter` method of the `HttpServletRequest` object. The response is written using the `getWriter` method of the `HttpServletResponse` object. The content type of the response is set using the `setContentType` method.

## Redirecting Requests to Other Resources in Servlets

In a servlet, you can redirect a request to another resource, such as a different servlet, JSP page, or HTML file, using the `sendRedirect` method of the `HttpServletResponse` object. Here is an example:

```

protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException {
    String newUrl = "http://www.example.com/newpage.html";
    response.sendRedirect(newUrl);
}

```

In this example, the `doGet` method of the servlet redirects the request to the URL specified in the `newUrl` variable. The `sendRedirect` method sends a temporary redirect response to the client, which causes the client to issue a new request to the specified URL. This is different from forwarding a request, where the request is forwarded to another resource within the same server without the client being aware of the change.

## Session Tracking in Servlets

Session tracking is a mechanism that servlets use to maintain state about a series of requests from the same user across some period of time. There are four techniques used for session tracking in servlets:

1. **Cookies:** A cookie is a small piece of information that is sent by a servlet to a Web browser, saved by the browser, and later sent back to the server. A cookie's value can uniquely identify a client, so cookies are commonly used for session management.
2. **Hidden Form Fields:** Hidden form fields are used when the client fills out an online form, and the form is submitted using either a GET or POST method. The form data is sent to the server-side script, which can then use hidden form fields to maintain the state of the user's session.
3. **URL Rewriting:** URL rewriting is a technique where the session ID is appended to the URL of the next servlet or the next resource to be accessed.
4. **HTTP Session:** An HTTP session is a way to store information on the server side, rather than on the client side. A session is created when a user logs in or adds an item to a shopping cart. The session information is stored on the server and is identified by a session ID.

Here is an example of how to create and use an HTTP session in a servlet:

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class SessionServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        HttpSession session = request.getSession();
        String name = (String) session.getAttribute("name");
        if (name == null) {
            name = request.getParameter("name");
            session.setAttribute("name", name);
        }
        out.println("Hello, " + name);
        out.close();
    }
}
```

This code creates an HTTP session and stores the user's name in the session. If the user's name is not already stored in the session, it is retrieved from the request parameter and stored in the session. The user's name is then retrieved from the session and displayed to the user. This allows the servlet to maintain state about the user's name across multiple requests.

## Cookies in Servlets

Here is an example of how to use cookies in servlets:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieExample extends HttpServlet {

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Create a new cookie
        Cookie cookie = new Cookie("username", "JohnDoe");

        // Set the maximum age of the cookie
        cookie.setMaxAge(60*60*24);

        // Add the cookie to the response
        response.addCookie(cookie);

        // Set the response content type
        response.setContentType("text/html");

        PrintWriter out = response.getWriter();
        String title = "Setting Cookies Example";
        String docType =
            "<!doctype html public \"-//w3c//dtd html 4.0 \" +
            \"transitional//en\">\n";

        out.println(docType +
            "<html>\n" +
            "<head><title>" + title + "</title></head>\n" +
            "<body bgcolor=\"#f0f0f0\">\n" +
            "<h1 align=\"center\">" + title + "</h1>\n" +
            "<ul>\n" +
            "    <li><b>Username</b>: "
            + request.getParameter("username") + "\n" +
            "</ul>\n" +
```



```

        "</body></html>");
    }
}

```

This code creates a new cookie with the name `username` and the value `JohnDoe`. The maximum age of the cookie is set to one day (60 seconds \* 60 minutes \* 24 hours). The cookie is then added to the response and sent to the client. The servlet also generates an HTML response that displays the value of the `username` parameter from the request.

### Session Tracking with Http Session in Servlets

Session tracking is a mechanism that enables you to maintain a user's state across multiple HTTP requests. One way to implement session tracking is by using the `HttpSession` object provided by the Servlet API.

Here is an example of how you can use `HttpSession` to track a user's session:

```

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class SessionServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        HttpSession session = request.getSession();
        String name = (String) session.getAttribute("name");
        if (name == null) {
            name = request.getParameter("name");
            session.setAttribute("name", name);
        }
        out.println("Hello, " + name);
        out.close();
    }
}

```

In the above example, we first check if the session already contains an attribute named “name”. If it does not, we retrieve the value of the “name” parameter from the request and store it in the session. Then, we use the value of the “name” attribute to generate a personalized greeting for the user.

## Java Server Pages (JSP) in Servlets

Java Server Pages (JSP) is a technology that helps software developers create dynamically generated web pages based on HTML, XML, or other document types. JSP is similar to PHP and ASP, but it uses the Java programming language.

Here is an example of a simple JSP page that displays the current time:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1" pageEncoding="ISO-8859-1"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Current Time</title>
</head>
<body>
<%
    java.util.Date date = new java.util.Date();
    out.println("<h2>Current Time: " + date.toString() + "</h2>");
%>
</body>
</html>
```

This JSP page can be deployed in a servlet container such as Apache Tomcat or Jetty. When a user accesses the page, the JSP code is compiled into a servlet and executed. The resulting HTML is then sent to the user's web browser.

## Introduction to JSP in Servlets

JavaServer Pages (JSP) is a technology that helps software developers create dynamically generated web pages based on HTML, XML, or other document types. JSP is similar to PHP and ASP, but it uses the Java programming language.

JSP pages are compiled into servlets by a JSP compiler. A JSP compiler is usually part of a web container, which is responsible for managing servlets and JSP pages. When a request is made for a JSP page, the web container checks if the page has already been compiled into a servlet. If it has not, the JSP compiler compiles the page into a servlet. The servlet is then executed and generates the response, which is sent back to the client.

Here is an example of a simple JSP page that displays the current time:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
<title>Current Time</title>
</head>
```

```

<body>
  <h1>The current time is <%= new java.util.Date() %></h1>
</body>
</html>

```

This page uses a JSP expression to insert the current time into the response. JSP expressions are enclosed in `<%= %>` and are evaluated at runtime. In this case, the expression `new java.util.Date()` creates a new `Date` object, which represents the current time. The `toString` method of the `Date` object is called implicitly to convert the date to a string, which is then inserted into the response.

JSP also provides other features, such as JSP directives, JSP actions, and custom tags, which allow developers to create more complex and dynamic web pages. These features will be discussed in more detail in later sections.

## Java Server Pages Overview in Servlets

Java Server Pages (JSP) is a technology that helps software developers create dynamically generated web pages based on HTML, XML, or other document types. JSP is similar to PHP and ASP, but it uses the Java programming language.

JSP pages are compiled into servlets by a JSP compiler. A JSP compiler is usually part of a web container, which is responsible for managing servlets and JSP pages. When a request is made for a JSP page, the web container checks if the page has been compiled into a servlet. If it has not, the JSP compiler compiles the page into a servlet. The servlet is then executed and generates the response that is sent back to the client.

Here is an example of a simple JSP page that displays the current date and time:

```

<%@ page import="java.util.*" %>
<html>
<head>
<title>Current Date and Time</title>
</head>
<body>
<h1>Current Date and Time</h1>
<%
    Date date = new Date();
    out.println(date.toString());
%>
</body>
</html>

```

This JSP page uses a scriptlet, which is a piece of Java code enclosed in `<%` and `%>` tags. The scriptlet creates a new `Date` object and uses the `out` object to print

the date to the response. The `out` object is an instance of `JspWriter`, which is a subclass of `java.io.Writer`. It is used to write content to the response.

JSP also provides several other elements, such as expressions, declarations, and directives, that can be used to create dynamic content. JSP pages can also include other files, such as HTML or JSP fragments, using the `<jsp:include>` element.

JSP is a powerful technology that can be used to create dynamic web pages. It is easy to learn and provides many features that make it a popular choice for web development.

### A First Java Server Page Example in Servlets

Here is an example of a simple Java Server Page (JSP) that can be used in a servlet:

```
<%@ page import="java.io.*,java.util.*" %>
<%
    String message = "Hello, World!";
%>
<html>
    <body>
        <h1><%= message %></h1>
    </body>
</html>
```

This JSP code imports the necessary Java classes, defines a `message` variable, and outputs the value of the `message` variable within an HTML `h1` element. When this JSP is accessed, it will display the text “Hello, World!” on the page.

### Implicit Objects in Servlets

In the context of Java Servlets, implicit objects are objects that are created by the container and are available to the developer without the need for explicit creation. These objects are available within the scope of a JSP page or a servlet and can be used to perform various tasks.

Here is a list of the implicit objects available in a servlet:

- **request**: an instance of `HttpServletRequest` representing the current request being processed by the servlet.
- **response**: an instance of `HttpServletResponse` representing the response being generated by the servlet.
- **out**: an instance of `PrintWriter` used to send output to the client.
- **session**: an instance of `HttpSession` representing the current session associated with the request.
- **application**: an instance of `ServletContext` representing the servlet context in which the servlet is running.

- `config`: an instance of `ServletConfig` representing the configuration information for the servlet.
- `pageContext`: an instance of `PageContext` representing the context in which the JSP page is being executed.

Here is an example of how these objects can be used in a servlet:

```
protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException {
    // Get the current session
    HttpSession session = request.getSession();

    // Get a parameter from the request
    String name = request.getParameter("name");

    // Set an attribute in the session
    session.setAttribute("name", name);

    // Get the servlet context
    ServletContext context = getServletContext();

    // Set an attribute in the application scope
    context.setAttribute("name", name);

    // Get the output writer
    PrintWriter out = response.getWriter();

    // Write some output
    out.println("Hello, " + name);
}
```

## Scripting in Servlets

Servlets can use scripting elements to generate dynamic content. Here is an example of a simple servlet that uses scripting elements to generate an HTML page:

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HelloWorld extends HttpServlet {
    public void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        out.println("<html>");
        out.println("<head><title>Hello World</title></head>");
    }
}
```

```

        out.println("<body>");
        out.println("<h1>Hello World</h1>");
        out.println("</body></html>");
    }
}

```

This servlet uses the `doGet` method to handle HTTP GET requests. The `response.setContentType` method is used to set the MIME type of the response. The `response.getWriter` method returns a `PrintWriter` object that can be used to send character data to the client. The servlet uses the `println` method of the `PrintWriter` object to generate the HTML page.

## Standard Actions in Servlets

Standard actions are predefined tags that are used to perform common tasks in JSP. These tags are provided by the JSP container and are used to manipulate the objects in the page context. Some of the standard actions in Servlets are:

- `<jsp:useBean>`: This action is used to create or locate a JavaBean object and make it available to the JSP page.
- `<jsp:setProperty>`: This action is used to set the properties of a JavaBean object.
- `<jsp:getProperty>`: This action is used to get the properties of a JavaBean object.
- `<jsp:include>`: This action is used to include the content of another resource, such as a JSP page or an HTML file, in the current JSP page.
- `<jsp:forward>`: This action is used to forward the request to another resource, such as a JSP page or a servlet.
- `<jsp:param>`: This action is used to pass parameters to the included or forwarded resource.
- `<jsp:plugin>`: This action is used to include a Java applet or a JavaBeans component in the JSP page.

Here is an example of using the `<jsp:useBean>` and `<jsp:setProperty>` actions in a JSP page:

```

<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<html>
<head>
    <title>Standard Actions Example</title>
</head>
<body>
    <jsp:useBean id="person" class="com.example.Person" />
    <jsp:setProperty name="person" property="name" value="John Doe" />
    <p>Name: <%= person.getName() %></p>
</body>
</html>

```

This code creates a `Person` object and sets its `name` property to “John Doe”. The value of the `name` property is then displayed on the page using a scriptlet.

## Directives in Servlets

Directives are instructions that are processed by the JSP engine when the page is compiled to a servlet. There are three types of directives: page, include, and taglib.

The page directive is used to provide instructions to the JSP engine that apply to the current JSP page. The syntax for the page directive is as follows:

```
<%@ page attribute="value" %>
```

Some common attributes of the page directive include: - `contentType`: Specifies the MIME type of the response generated by the JSP page. - `import`: Specifies the classes or packages that should be imported for use in the JSP page. - `errorPage`: Specifies the URL of the error page to which the JSP engine should forward the request if an unhandled exception occurs while processing the JSP page.

The include directive is used to include the contents of another file in the current JSP page. The syntax for the include directive is as follows:

```
<%@ include file="filename" %>
```

The taglib directive is used to specify a tag library that should be used in the JSP page. The syntax for the taglib directive is as follows:

```
<%@ taglib uri="uri" prefix="prefix" %>
```

The `uri` attribute specifies the location of the tag library descriptor, and the `prefix` attribute specifies the prefix that should be used to reference the tags defined in the tag library.

## Custom Tag Libraries in Servlets

- Custom tag libraries are a feature of JavaServer Pages (JSP) technology that allows developers to create their own custom tags for use in JSP pages.
- Custom tags are used to encapsulate reusable functionality and can be used to simplify the development of JSP pages.
- Custom tag libraries are defined in a Tag Library Descriptor (TLD) file, which specifies the tags and their attributes, as well as any scripting variables or functions that the tags may use.
- Custom tags can be used to perform a variety of tasks, such as generating dynamic content, accessing databases, and performing calculations.
- To use a custom tag library in a JSP page, the taglib directive must be included at the top of the page, specifying the URI of the TLD file.

- Custom tag libraries can be used to promote code reuse and modularization, making it easier to maintain and update JSP pages.
- Custom tag libraries can also be used to create a consistent look and feel across multiple JSP pages, by encapsulating common page elements such as headers and footers in custom tags.
- Custom tag libraries can be developed by individual developers or shared among a team, allowing for collaboration and code sharing.
- Custom tag libraries can be used in conjunction with other JSP technologies, such as expression language and standard tag libraries, to create powerful and dynamic web applications.